TIP If you don't want to add a prefix but still want to use the route group for applying filters, you can pass the prefix `"/"` to `MapGroup()`.

Listing 5.13 shows an example of rewriting the `fruit` API to use route groups. It creates a top-level `fruitApi`, which applies the `"/fruit"` prefix, and creates a nested route group called `fruitApiWithValidation` for the endpoints that require a filter. You can find the complete example comparing the versions with and without route groups in the source code for this chapter.

Listing 5.13 Reducing duplication with route groups

```
using System.Collections.Concurrent;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

var _fruit = new ConcurrentDictionary<string, Fruit>();

RouteGroupBuilder fruitApi = app.MapGroup("/fruit");                    ❶

fruitApi.MapGet("/", () => _fruit);                                     ❷

RouteGroupBuilder fruitApiWithValidation = fruitApi.MapGroup("/")       ❸
    .AddEndpointFilter(ValidationHelper.ValidateIdFactory);             ❹

fruitApiWithValidation.MapGet("/{id}", (string id) =>                   ❺
    _fruit.TryGetValue(id, out var fruit)
        ? TypedResults.Ok(fruit)
        : Results.Problem(statusCode: 404));

fruitApiWithValidation.MapPost("/{id}", (Fruit fruit, string id) =>     ❺
    _fruit.TryAdd(id, fruit)
        ? TypedResults.Created($"/fruit/{id}", fruit)
        : Results.ValidationProblem(new Dictionary<string, string[]>
          {
              { "id", new[] { "A fruit with this id already exists" } }
          }));

fruitApiWithValidation.MapPut("/{id}", (string id, Fruit fruit) =>      ❺
{
    _fruit[id] = fruit;
```